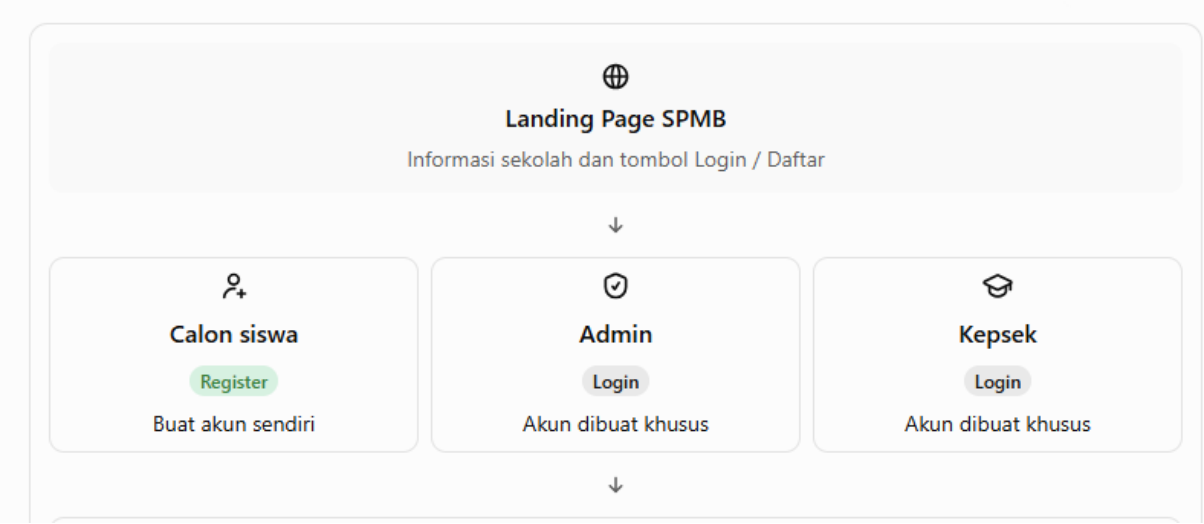
Materi Praktik Project SPMB Next.js + Laragon — 3 Role Pengguna

1. Alur 3 role dalam aplikasi SPMB





Catatan: fungsi dashboard di atas merupakan rancangan awal yang bisa disesuaikan dengan kebutuhan sekolah.

Tahap 1 — Membuat tabel users untuk 3 role

Karena tabel users belum dibuat, kita mulai dari sini.

Buka HeidiSQL atau phpMyAdmin melalui Laragon, pilih database project SPMB yang sudah terhubung, lalu jalankan SQL berikut.

SQL

```sql
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nama VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role ENUM('calon_siswa', 'admin', 'kepsek')
        NOT NULL DEFAULT 'calon_siswa',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Penjelasan untuk siswa

Kolom	Fungsi
id	Identitas unik akun
nama	Nama pengguna
email	Email untuk login
password	Password yang sudah di-hash
role	Jenis pengguna
created_at	Tanggal akun dibuat

Perbedaan dengan tabel sebelumnya adalah kolom role sekarang memiliki tiga pilihan: calon_siswa, admin, dan kepsek.

Tugas praktik: jalankan DESCRIBE users; dan pastikan kolom role sudah memiliki ketiga pilihan tersebut.

Jika tabel users telanjur dibuat dengan enum dua pilihan, jangan jalankan CREATE TABLE lagi. Gunakan perintah perubahan kolom berikut:

```sql
ALTER TABLE users
MODIFY role ENUM('calon_siswa', 'admin', 'kepsek')
NOT NULL DEFAULT 'calon_siswa';
```

Tahap 2 — Membuat Register khusus calon siswa

Halaman Register yang sudah dibuat siswa akan digunakan untuk calon siswa saja.

Alurnya:

1. Calon siswa mengisi nama, email, dan password.
2. Frontend mengirim data ke API Register.
3. API memvalidasi input.
4. API melakukan hash password.
5. API menyimpan akun dengan role calon_siswa.

   Admin dan kepsek tidak boleh menentukan role sendiri dari form Register.

A. Instal package

Di terminal project:

npm install bcryptjs

Pastikan file koneksi database lib/db.js yang sudah berfungsi tetap digunakan.

B. Buat API Register

Buat file app/api/register/route.js.

```
import pool from "@/lib/db";
import bcrypt from "bcryptjs";


export async function POST(request) {
 try {
  const body = await request.json();


  const nama = body.nama?.trim();
  const email = body.email?.trim().toLowerCase();
  const password = body.password;


  if (
   !nama ||
   !email ||
   typeof password !== "string"
  ) {
   return Response.json(
    { message: "Semua kolom wajib diisi." },
    { status: 400 }
   );
```

```
}

if (nama.length > 100 || email.length > 150) {
  return Response.json(
    { message: "Nama atau email terlalu panjang." },
    { status: 400 }
  );
}

if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
  return Response.json(
    { message: "Format email tidak valid." },
    { status: 400 }
  );
}

if (
  password.length < 8 ||
  Buffer.byteLength(password, "utf8") > 72
) {
  return Response.json(
    { message: "Password minimal 8 karakter." },
    { status: 400 }
  );
}

const [existing] = await pool.execute(
  "SELECT id FROM users WHERE email = ?",
```

```
    [email]
  );

  if (existing.length > 0) {
    return Response.json(
      { message: "Email sudah terdaftar." },
      { status: 409 }
    );
  }

  const hashedPassword = await bcrypt.hash(password, 12);

  // Role ditentukan oleh server, bukan oleh form.
  await pool.execute(
    `INSERT INTO users (nama, email, password, role)
     VALUES (?, ?, ?, 'calon_siswa')`,
    [nama, email, hashedPassword]
  );

  return Response.json(
    { message: "Registrasi calon siswa berhasil." },
    { status: 201 }
  );
} catch (error) {
  if (error.code === "ER_DUP_ENTRY") {
    return Response.json(
      { message: "Email sudah terdaftar." },
      { status: 409 }
```

```
    );
  }

  console.error("Register error:", error);

  return Response.json(
    { message: "Terjadi kesalahan pada server." },
    { status: 500 }
  );
 }
}
```

Mengapa role ditentukan oleh server?

Misalnya, ada calon siswa yang mengubah request dari browser menjadi:

```json
{
  "nama": "Siswa Baru",
  "email": "siswa@example.com",
  "password": "BelajarSPMB123",
  "role": "admin"
}
```

API di atas tetap menyimpan akun sebagai calon_siswa, karena nilai role dalam request tidak digunakan.

Ini merupakan konsep keamanan penting yang harus dipahami siswa.

C. Hubungkan form Register

Di halaman app/register/page.js, pertahankan desain Figma. Tambahkan logika pengiriman form seperti berikut:

JavaScript

```javascript
async function handleSubmit(e) {
  e.preventDefault();

  const response = await fetch("/api/register", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      nama,
      email,
      password,
    }),
  });

  const data = await response.json();

  if (response.ok) {
    alert(data.message);
    // Arahkan ke halaman Login.
  } else {
    alert(data.message);
  }
}
```

Kode tersebut mengasumsikan state nama, email, dan password sudah dibuat. Siswa dapat menggunakan fungsi handleSubmit yang lengkap dari materi sebelumnya apabila state form mereka berbentuk object.

Target Tahap 2: calon siswa dapat mendaftar, datanya tersimpan di MySQL, dan role otomatis menjadi calon_siswa.

Tahap 3 — Membuat akun Admin dan Kepsek

Admin dan kepsek hanya bisa login. Mereka tidak menggunakan Register publik.

Ada beberapa cara membuat akun awal:

- Melalui seeder atau script khusus yang dijalankan oleh pengelola.
- Melalui fitur manajemen akun internal yang hanya dapat diakses oleh pihak berwenang.

Untuk praktik awal, gunakan script khusus agar siswa memahami bagaimana akun dengan role berbeda disiapkan.

Buat file sementara scripts/create-user.js:

```
import pool from "../lib/db.js";
import bcrypt from "bcryptjs";

async function createUser() {
 const nama = process.argv[2];
 const email = process.argv[3]?.toLowerCase();
 const role = process.argv[4];
 const password = process.env.INITIAL_USER_PASSWORD;

 if (
  !nama ||
  !email ||
  !["admin", "kepsek"].includes(role) ||
  !password ||
  password.length < 8 ||
  Buffer.byteLength(password, "utf8") > 72
 ) {
  throw new Error(
   "Isi nama, email, role admin/kepsek, dan password yang valid."
  );
 }

 const hashedPassword = await bcrypt.hash(password, 12);

 await pool.execute(
  `INSERT INTO users (nama, email, password, role)
```

```
    VALUES (?, ?, ?, ?)`,
    [nama, email, hashedPassword, role]
  );

  console.log(`Akun ${role} berhasil dibuat.`);
}

createUser()
  .catch((error) => {
    console.error("Gagal membuat akun:", error.message);
    process.exitCode = 1;
  })
  .finally(() => pool.end());
```

Script ini menggunakan koneksi yang sama dengan aplikasi. Karena dijalankan sebagai script Node.js, pastikan konfigurasi environment dimuat sebelum menjalankannya dan sesuaikan import lib/db.js dengan konfigurasi project.

Jalankan script satu kali untuk setiap akun, menggunakan password sementara yang kuat dan disimpan melalui environment, bukan ditulis langsung di kode. Hapus atau amankan script setelah proses pembuatan akun selesai.

Contoh hasil akun yang diharapkan:

Nama akun	**Role**
Petugas SPMB	admin
Kepala Sekolah	kepsek
Siswa Percobaan	calon_siswa

Jangan memberikan kredensial admin atau kepsek kepada calon siswa. Di aplikasi nyata, akun tersebut sebaiknya disiapkan dan dikelola oleh pihak sekolah.

Tahap 4 — Membuat satu API Login untuk semua role

Kita tidak perlu membuat tiga halaman Login yang berbeda. Cukup satu halaman Login untuk semua pengguna.

API akan memeriksa email dan password, kemudian membaca role dari database. Role tidak boleh dipercaya jika dikirim oleh browser.

A. Instal package sesi

Jalankan:

Bash

```
npm install jose
```

Tambahkan secret acak yang kuat ke `.env.local` :

Bash

```
SESSION_SECRET=isi_dengan_secret_acak_yang_kuat
```

Gunakan secret yang benar-benar acak, jangan gunakan contoh di atas sebagai nilai produksi. Restart server Next.js setelah mengubah environment.

B. Buat API Login

Buat file app/api/login/route.js.

```
import pool from "@/lib/db";
import bcrypt from "bcryptjs";
import { SignJWT } from "jose";

export async function POST(request) {
 try {
  const body = await request.json();
  const email = body.email?.trim().toLowerCase();
  const password = body.password;

  if (
   !email ||
```

```
  typeof password !== "string" ||
  !password
) {
  return Response.json(
    { message: "Email dan password wajib diisi." },
    { status: 400 }
  );
}

const [rows] = await pool.execute(
  `SELECT id, nama, email, password, role
   FROM users
   WHERE email = ?`,
  [email]
);

const user = rows[0];

if (
  !user ||
  !(await bcrypt.compare(password, user.password))
) {
  return Response.json(
    { message: "Email atau password salah." },
    { status: 401 }
  );
}

const secret = process.env.SESSION_SECRET;

if (!secret || secret.length < 32) {
```

```
  return Response.json(
    { message: "Konfigurasi sesi belum tersedia." },
    { status: 500 }
  );
}

const token = await new SignJWT({
  nama: user.nama,
  role: user.role,
})
  .setProtectedHeader({ alg: "HS256" })
  .setSubject(String(user.id))
  .setIssuedAt()
  .setExpirationTime("8h")
  .sign(new TextEncoder().encode(secret));

const cookie = [
  `spmb_session=${token}`,
  "HttpOnly",
  "Path=/",
  "SameSite=Lax",
  "Max-Age=28800",
];

if (process.env.NODE_ENV === "production") {
  cookie.push("Secure");
}

const response = Response.json({
  message: "Login berhasil.",
  user: {
```

```
      id: user.id,
      nama: user.nama,
      role: user.role,
    },
  });

  response.headers.append(
    "Set-Cookie",
    cookie.join("; ")
  );

  return response;
 } catch (error) {
  console.error("Login error:", error);

  return Response.json(
    { message: "Terjadi kesalahan pada server." },
    { status: 500 }
  );
 }
}
```

Alur API Login

1. Pengguna memasukkan email dan password.
2. Server mencari email di database.
3. Server memeriksa password dengan bcrypt.compare().
4. Jika benar, server membaca role akun dari database.
5. Server membuat cookie sesi yang tidak bisa dibaca JavaScript browser.
6. Frontend menerima informasi role untuk menentukan halaman tujuan.

Tahap 5 — Mengarahkan pengguna sesuai role

Tambahkan logika berikut ke fungsi handleSubmit di halaman Login yang sudah dibuat siswa.

```
import { useRouter } from "next/navigation";

// Di dalam component:
const router = useRouter();

async function handleSubmit(e) {
 e.preventDefault();

 const response = await fetch("/api/login", {
  method: "POST",
  headers: {
   "Content-Type": "application/json",
  },
  body: JSON.stringify({ email, password }),
 });

 const data = await response.json();

 if (!response.ok) {
  setPesan(data.message || "Login gagal.");
  return;
 }

 if (data.user.role === "calon_siswa") {
  router.push("/dashboard/calon-siswa");
 } else if (data.user.role === "admin") {
  router.push("/dashboard/admin");
 } else if (data.user.role === "kepsek") {
```

```
    router.push("/dashboard/kepsek");
  } else {
    setPesan("Role akun tidak dikenali.");
  }

  router.refresh();
}
```

Kode ini mengasumsikan halaman Login sudah memiliki state email, password, dan setPesan. Letakkan useRouter() di dalam komponen React, bukan di luar komponen.

Penting: pengalihan halaman berdasarkan role hanya mengatur navigasi. Keamanan sebenarnya harus tetap dilakukan oleh server pada setiap halaman dan API privat.

Tahap 6 — Membatasi akses berdasarkan role

Misalnya, calon siswa mencoba mengetik URL /dashboard/admin secara langsung. Sistem harus menolak akses tersebut, meskipun pengguna mengetahui alamat halamannya.

A. Buat helper autentikasi

Buat lib/auth.js:

```
import { jwtVerify } from "jose";

export async function getSession(token) {
  const secret = process.env.SESSION_SECRET;

  if (!token || !secret || secret.length < 32) {
    return null;
```

```
  }

  try {
    const { payload } = await jwtVerify(
      token,
      new TextEncoder().encode(secret),
      { algorithms: ["HS256"] }
    );

    return payload;
  } catch {
    return null;
  }
}
```

B. Buat dashboard sesuai role

Struktur file:

```
app/
└── dashboard/
    ├── calon-siswa/
    │   └── page.js
    ├── admin/
    │   └── page.js
    └── kepsek/
        └── page.js
```

TAHAP SELANJUTNYA DILAKUKAN SENDIRI.

Tahap 7 — Tugas praktik siswa

Checklist penilaian

- [ ] **Database**
  Tabel users memiliki tiga pilihan role.
- [ ] **Register**
  Pengguna publik hanya bisa membuat akun calon_siswa.
- [ ] **Akun internal**
  Admin dan kepsek dibuat oleh pengelola.
- [ ] **Login**
  Satu halaman login bisa memproses ketiga role.
- [ ] **Navigasi**
  Setiap role diarahkan ke dashboard yang benar.
- [ ] **Proteksi**
  Calon siswa tidak bisa membuka dashboard admin atau kepsek.

Deadline: 13/10/2026 07.00AM